

GUÍA DE LABORATORIO 3

Pruebas de Integración del Controlador – notes-api

Proyecto: notes-api-db

Materia: Verificación y Validación de Software (INF780)

Nivel: Universitario | Duración: 3 horas

Herramientas: NestJS · Jest · TypeScript

2026

1. Objetivos

Al finalizar esta guía el estudiante será capaz de:

- Configurar el módulo de pruebas de NestJS para aislar el controlador mediante un servicio mock.
- Escribir pruebas de integración que verifiquen la delegación correcta del controlador al servicio.
- Validar el comportamiento del pipe `ParseUUIDPipe` ante entradas válidas e inválidas.
- Aplicar `class-transformer` y `class-validator` para comprobar las reglas de validación de los DTOs.
- Distinguir entre pruebas unitarias del servicio (Guía 2) y pruebas de integración del controlador (esta guía).
- Ejecutar las pruebas y analizar el reporte de cobertura del controlador.

2. Conceptos Clave

2.1 Prueba unitaria vs. prueba de integración del controlador

En la Guía 2 se probó el **servicio** de forma aislada, reemplazando el repositorio por un mock. En esta guía se prueba el **controlador**, que es la capa que recibe las peticiones HTTP y delega al servicio. La diferencia fundamental es el **sujeto bajo prueba** (SUT): ahora el controlador es el SUT y el servicio es la dependencia simulada.

DEF

Prueba de integración del controlador: verifica que el controlador invoca correctamente los métodos del servicio, transforma los parámetros de entrada (query params, DTOs) y retorna los valores esperados. No levanta un servidor HTTP real; usa `Test.createTestingModule()` de NestJS para compilar el módulo con dependencias simuladas.

2.2 MockService: el patrón aplicado al controlador

El patrón es idéntico al de la Guía 2, pero ahora se simula el servicio en lugar del repositorio. Se define un tipo `MockService` con un mapped type que convierte cada método de `NotesService` en `jest.Mock`, y una función `factory createMockService()` que genera una instancia fresca antes de cada prueba.

2.3 ParseUUIDPipe: validación de parámetros

NestJS proporciona el pipe `ParseUUIDPipe` para validar que un parámetro de ruta sea un UUID válido. Si el valor no cumple el formato UUID, el pipe lanza una `BadRequestException` antes de que el controlador procese la petición. En las pruebas se instancia el pipe directamente y se verifica su comportamiento con entradas válidas e inválidas.

2.4 Validación de DTOs con class-validator

Los DTOs (`CreateNoteDto`, `UpdateNoteDto`) usan decoradores de `class-validator` como `@IsString()`, `@IsNotEmpty()`, `@MaxLength(200)` y `@IsOptional()`. Para probarlos fuera del contexto HTTP se usa `plainToInstance()` de `class-transformer` para crear la instancia del DTO y luego `validate()` de `class-validator` para ejecutar las reglas de validación.

NOTA

Este enfoque permite verificar las reglas de validación de forma aislada sin necesidad de levantar un servidor HTTP ni enviar peticiones reales. Es una técnica especialmente útil cuando el proyecto crece y los DTOs acumulan reglas de validación complejas.

2.5 Matchers adicionales en esta guía

Matcher	Qué verifica
toBeGreaterThan(n)	Que el valor es estrictamente mayor que n.
some(fn)	Que al menos un elemento del array cumple la condición fn. Se usa para buscar errores por propiedad en el array de validación.
toBe(valor)	Igualdad estricta por referencia o valor primitivo.
toHaveLength(n)	Longitud de un array o string. Cuando se espera que no haya errores de validación: toHaveLength(0).

3. Desarrollo

El repositorio del proyecto está disponible en:

<https://github.com/HuascarFedor/INF780-NotesApiDb>

Si ya clonó el repositorio en la Guía 2, ingrese al directorio y asegúrese de tener las dependencias instaladas. Si es la primera vez, clone e instale:

```
git clone https://github.com/HuascarFedor/INF780-NotesApiDb.git
cd INF780-NotesApiDb
npm install

# El archivo de pruebas va en:
src/notes/notes.controller.spec.ts
```

1 Importaciones y datos de prueba

Abra el archivo `src/notes/notes.controller.spec.ts` (créelo si no existe) y agregue las importaciones. Note que ahora se importan `BadRequestException`, `ParseUUIDPipe`, `plainToInstance` y `validate`, que no se usaron en la Guía 2.

```
import { BadRequestException } from '@nestjs/common';
import { ParseUUIDPipe } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { plainToInstance } from 'class-transformer';
import { validate } from 'class-validator';
import { NotesController } from '../notes.controller';
import { NotesService } from '../notes.service';
import { Note } from '../entities/note.entity';
import { CreateNoteDto } from '../dto/create-note.dto';
import { UpdateNoteDto } from '../dto/update-note.dto';

const mockNote: Note = {
  id: '123e4567-e89b-12d3-a456-426614174000',
```

```
title: 'Test Note',
content: 'Test content',
isArchived: false,
createdAt: new Date('2024-01-01T00:00:00.000Z'),
updatedAt: new Date('2024-01-01T00:00:00.000Z'),
};
```

NOTA

Se reutiliza el mismo fixture `mockNote` de la Guía 2. Al mantener los mismos datos de prueba en ambas guías, se facilita la comparación de resultados entre las pruebas del servicio y las del controlador.

2 Tipo MockService y función factory

Defina el tipo `MockService` y la función `createMockService()`. El patrón es idéntico al de la Guía 2, pero ahora simula `NotesService` en lugar de `NotesRepository`.

```
type MockService = {
  [K in keyof NotesService]: jest.Mock;
};

const createMockService = (): MockService => ({
  create: jest.fn(),
  findAll: jest.fn(),
  findOne: jest.fn(),
  update: jest.fn(),
  remove: jest.fn(),
});
```

DEF

La diferencia con la Guía 2 es semántica, no estructural: el mapped type ahora recorre las claves de `NotesService` (no de `NotesRepository`). Si el servicio agrega un método nuevo, TypeScript exigirá que el mock también lo incluya.

3 Configuración del módulo de pruebas

Defina el bloque `describe` principal. A diferencia de la Guía 2, aquí se registra el controlador en `controllers` y el servicio mock en `providers`.

```
describe('NotesController', () => {
  let controller: NotesController;
  let service: MockService;

  beforeEach(async () => {
    service = createMockService();

    const module: TestingModule = await Test.createTestingModule({
      controllers: [NotesController],
      providers: [{ provide: NotesService, useValue: service }],
    }).compile();
```

```
    controller = module.get<NotesController>(NotesController);
  });

  afterEach(() => {
    jest.clearAllMocks();
  });

  // ... bloques describe por método
});
```

La clave está en la línea `{ provide: NotesService, useValue: service }`: le indica al módulo que cuando el controlador solicite `NotesService` como dependencia, debe entregar el objeto mock en lugar del servicio real. Esto aísla completamente al controlador de la lógica de negocio y de la base de datos.

4 Pruebas de create — 1 caso

El método `create()` del controlador recibe un `CreateNoteDto` y delega al servicio. La prueba verifica que: (a) el servicio fue llamado con el DTO correcto, (b) fue llamado exactamente una vez, y (c) el valor devuelto es `mockNote`.

```
describe('create', () => {
  it('should invoke notesService.create with the received DTO', async () => {
    const dto: CreateNoteDto = { title: 'New Note', content: 'Some content' };
    service.create.mockResolvedValue(mockNote);

    const result = await controller.create(dto);

    expect(service.create).toHaveBeenCalledWith(dto);
    expect(service.create).toHaveBeenCalledTimes(1);
    expect(result).toEqual(mockNote);
  });
});
```

5 Pruebas de findAll — 4 casos

`findAll()` recibe un `query param` opcional de tipo string (`"true"`, `"false"` o cualquier otro valor). El controlador convierte el string a booleano antes de pasarlo al servicio. Se necesitan cuatro casos: sin parámetro, `"true"`, `"false"` y un valor no reconocido.

```
describe('findAll', () => {
  it('should invoke notesService.findAll with undefined when no query param',
  async () => {
    service.findAll.mockResolvedValue([mockNote]);

    const result = await controller.findAll();

    expect(service.findAll).toHaveBeenCalledWith(undefined);
  });
});
```

```
    expect(result).toEqual([mockNote]);
  });

  it('should pass archived=true when query param is "true"', async () => {
    service.findAll.mockResolvedValue([]);

    await controller.findAll('true');

    expect(service.findAll).toHaveBeenCalledWith(true);
  });

  it('should pass archived=false when query param is "false"', async () => {
    service.findAll.mockResolvedValue([mockNote]);

    await controller.findAll('false');

    expect(service.findAll).toHaveBeenCalledWith(false);
  });

  it('should pass undefined when query param is an unrecognized value', async ()
=> {
    service.findAll.mockResolvedValue([mockNote]);

    await controller.findAll('maybe');

    expect(service.findAll).toHaveBeenCalledWith(undefined);
  });
});
```

NOTA

El cuarto caso (valor no reconocido como 'maybe') es esencial: verifica que el controlador no convierte cadenas arbitrarias a `true` por truthy coercion, sino que las trata como `undefined`. Esto protege contra un error sutil de JavaScript.

6 Pruebas de findOne — 1 caso

`findOne()` recibe un `id` y lo pasa directamente al servicio. A diferencia de la Guía 2 (donde se probaba la excepción en el servicio), aquí solo se verifica la delegación. La validación del UUID se prueba por separado con `ParseUUIDPipe`.

```
describe('findOne', () => {
  it('should invoke notesService.findOne with the given id', async () => {
    service.findOne.mockResolvedValue(mockNote);

    const result = await controller.findOne(mockNote.id);

    expect(service.findOne).toHaveBeenCalledWith(mockNote.id);
    expect(result).toEqual(mockNote);
  });
});
```

7 Pruebas de update — 1 caso

`update()` recibe un `id` y un `UpdateNoteDto`. Se verifica que el controlador pasa ambos argumentos al servicio y devuelve la nota actualizada.

```
describe('update', () => {
  it('should invoke notesService.update with id and DTO', async () => {
    const dto: UpdateNoteDto = { title: 'Updated Title' };
    const updatedNote: Note = { ...mockNote, title: 'Updated Title' };
    service.update.mockResolvedValue(updatedNote);

    const result = await controller.update(mockNote.id, dto);

    expect(service.update).toHaveBeenCalledWith(mockNote.id, dto);
    expect(result).toEqual(updatedNote);
  });
});
```

8 Pruebas de remove — 1 caso

`remove()` delega la eliminación al servicio. Se verifica que fue llamado con el `id` correcto y exactamente una vez.

```
describe('remove', () => {
  it('should invoke notesService.remove with the given id', async () => {
    service.remove.mockResolvedValue(undefined);

    await controller.remove(mockNote.id);

    expect(service.remove).toHaveBeenCalledWith(mockNote.id);
    expect(service.remove).toHaveBeenCalledTimes(1);
  });
});
```

9 Pruebas de ParseUUIDPipe — 3 casos

Estas pruebas validan el comportamiento del pipe `ParseUUIDPipe` de forma aislada. Se instancia directamente y se invoca su método `transform()` con distintas entradas.

```
describe('ParseUUIDPipe', () => {
  it('should reject invalid UUIDs with BadRequestException', async () => {
    const pipe = new ParseUUIDPipe();

    await expect(pipe.transform('not-a-uuid', { type: 'param'
    })).rejects.toThrow(
      BadRequestException,
    );
  });

  it('should accept and return a valid UUID v4', async () => {
    const pipe = new ParseUUIDPipe();
```

```
const validUuid = '123e4567-e89b-12d3-a456-426614174000';

const result = await pipe.transform(validUuid, { type: 'param' });

expect(result).toBe(validUuid);
});

it('should reject an empty string', async () => {
  const pipe = new ParseUUIDPipe();

  await expect(pipe.transform('', { type: 'param'
})).rejects.toThrow(BadRequestException);
});
});
```

DEF

El segundo argumento de `pipe.transform()` es un objeto `ArgumentMetadata` que indica el origen del valor. Con `{ type: 'param' }` se simula que el valor proviene de un parámetro de ruta (`@Param()`). Aunque en estas pruebas el pipe no varía su comportamiento por tipo, es buena práctica especificarlo.

10 Pruebas de validación de DTOs — 7 casos

Estas pruebas verifican las reglas de validación de los DTOs sin necesidad de un servidor HTTP. Se usa `plainToInstance()` para crear instancias y `validate()` para ejecutar los decoradores.

CreateNoteDto — 5 casos:

```
describe('DTO validation', () => {
  describe('CreateNoteDto', () => {
    it('should fail when title is missing', async () => {
      const dto = plainToInstance(CreateNoteDto, { content: 'Some content' });
      const errors = await validate(dto);

      expect(errors.length).toBeGreaterThan(0);
      expect(errors.some((e) => e.property === 'title')).toBe(true);
    });

    it('should fail when content is missing', async () => {
      const dto = plainToInstance(CreateNoteDto, { title: 'Valid title' });
      const errors = await validate(dto);

      expect(errors.length).toBeGreaterThan(0);
      expect(errors.some((e) => e.property === 'content')).toBe(true);
    });

    it('should fail when title exceeds 200 characters', async () => {
      const dto = plainToInstance(CreateNoteDto, {
        title: 'a'.repeat(201),
        content: 'Valid content',
      });
      const errors = await validate(dto);
```



```
    expect(errors.length).toBeGreaterThan(0);
    expect(errors.some((e) => e.property === 'title')).toBe(true);
  });

  it('should pass with valid title and content', async () => {
    const dto = plainToInstance(CreateNoteDto, {
      title: 'Valid title',
      content: 'Valid content',
    });
    const errors = await validate(dto);

    expect(errors).toHaveLength(0);
  });

  it('should pass with optional isArchived set to true', async () => {
    const dto = plainToInstance(CreateNoteDto, {
      title: 'Valid title',
      content: 'Valid content',
      isArchived: true,
    });
    const errors = await validate(dto);

    expect(errors).toHaveLength(0);
  });
});
```

UpdateNoteDto — 2 casos:

```
describe('UpdateNoteDto', () => {
  it('should pass with an empty object (all fields optional)', async () => {
    const dto = plainToInstance(UpdateNoteDto, {});
    const errors = await validate(dto);

    expect(errors).toHaveLength(0);
  });

  it('should fail when title exceeds 200 characters', async () => {
    const dto = plainToInstance(UpdateNoteDto, { title: 'x'.repeat(201) });
    const errors = await validate(dto);

    expect(errors.length).toBeGreaterThan(0);
  });
});
```

NOTA

El caso de `UpdateNoteDto` con objeto vacío es crucial: demuestra que todos los campos son opcionales en una actualización parcial, lo cual es el comportamiento esperado de un `PATCH`. Si el DTO exigiera campos obligatorios, este caso fallaría y revelaría un defecto en la definición del DTO.

11 Ejecutar las pruebas y verificar cobertura

Desde la raíz del proyecto ejecute los siguientes comandos. Todos los casos deben pasar en verde.

```
# Ejecutar solo las pruebas del controlador
npm run test -- notes.controller

# Modo watch (re-ejecuta al guardar el archivo)
npm run test:watch -- notes.controller

# Con reporte de cobertura
npm run test:cov
```

La salida esperada al ejecutar las pruebas del controlador es:

```
PASS src/notes/notes.controller.spec.ts
  NotesController
    create
      ✓ should invoke notesService.create with the received DTO
    findAll
      ✓ should invoke notesService.findAll with undefined when no query param
      ✓ should pass archived=true when query param is "true"
      ✓ should pass archived=false when query param is "false"
      ✓ should pass undefined when query param is an unrecognized value
    findOne
      ✓ should invoke notesService.findOne with the given id
    update
      ✓ should invoke notesService.update with id and DTO
    remove
      ✓ should invoke notesService.remove with the given id
    ParseUUIDPipe
      ✓ should reject invalid UUIDs with BadRequestException
      ✓ should accept and return a valid UUID v4
      ✓ should reject an empty string
    DTO validation
      CreateNoteDto
        ✓ should fail when title is missing
        ✓ should fail when content is missing
        ✓ should fail when title exceeds 200 characters
        ✓ should pass with valid title and content
        ✓ should pass with optional isArchived set to true
      UpdateNoteDto
        ✓ should pass with an empty object (all fields optional)
        ✓ should fail when title exceeds 200 characters

Tests:      18 passed, 18 total
```

El reporte de cobertura del controlador debe mostrar 100% en todas las columnas:

File	% Stmts	% Branch	% Funcs	% Lines
notes/notes.controller.ts	100	100	100	100

4. Rúbrica de Evaluación

Criterio	Indicadores	Pts	Obtenido
Fixture y tipos de soporte	mockNote con los 6 campos; MockService con mapped type; createMockService() con 5 jest.fn()	10	
Configuración del módulo	beforeEach registra controlador en controllers y servicio mock en providers; afterEach limpia con clearAllMocks	10	
Pruebas de create (1 caso)	Verifica toHaveBeenCalledWith, toHaveBeenCalledTimes(1) y valor de retorno	10	
Pruebas de findAll (4 casos)	Cubre undefined, "true", "false" y valor no reconocido; verifica argumento pasado al servicio	15	
Pruebas de findOne (1 caso)	Verifica delegación al servicio con el id correcto	5	
Pruebas de update (1 caso)	Verifica delegación con id y DTO; valor de retorno correcto	10	
Pruebas de remove (1 caso)	Verifica toHaveBeenCalledWith y toHaveBeenCalledTimes(1)	5	
ParseUUIDPipe (3 casos)	Rechaza UUID inválido y string vacío con BadRequestException; acepta UUID v4 válido	10	
Validación CreateNoteDto (5 casos)	Fallo por title faltante, content faltante, title > 200 chars; éxito con datos válidos y con isArchived opcional	10	
Validación UpdateNoteDto (2 casos)	Objeto vacío pasa (campos opcionales); title > 200 chars falla	10	
Cobertura 100%	npm run test:cov reporta 100% en todas las columnas de notes.controller.ts	5	
TOTAL		100	

IMPORTANTE

Estas pruebas de integración verifican que el controlador delega correctamente al servicio y que los mecanismos de validación (pipes y DTOs) funcionan según lo esperado. No prueban la lógica de negocio del servicio (cubierta en la Guía 2) ni la integración con la base de datos. Las pruebas end-to-end (e2e) que cubren toda la pila se abordará en la guía posterior.